



STUDENT ASSIGNMENT HANDOUT

Content Management System (CMS) for Students

Project Title	Tech Stack	Difficulty Level
Content Management System (CMS) for Students	React (Frontend), Laravel (Backend), MySQL (Database)	Intermediate–Advanced (Production-Level)



Objectives

- Build a full-stack application using React, Laravel, and MySQL.
- Understand role-based access control (Admin, Teacher, Student).
- Implement content management with CRUD operations.
- Practice authentication, authorization, and API integration.
- Deliver a production-style project with proper structure and documentation.



Project Requirements

Authentication

- Login & registration (Laravel Sanctum).
- Role-based access (Admin, Teacher, Student).

User Management

- Admin can create/manage users.
- Teacher can manage content and assignments.
- Student can view content and submit assignments.

Content Management

- CRUD for notes, announcements, resources.
- Categories (subject-wise).
- Draft vs Published states.

Assignments

- Teacher creates assignments with deadlines.
- Students upload submissions (PDF/DOCX).
- Teacher grades submissions.

Notifications

- Show deadlines, new content, grades.



Database Schema (Simplified)

-- Users

```

CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(150) UNIQUE,
  password VARCHAR(255),
  role ENUM('admin','teacher','student'),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

-- Content
CREATE TABLE contents (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255),
  body TEXT,
  category VARCHAR(100),
  author_id BIGINT,
  status ENUM('draft','published'),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

-- Assignments
CREATE TABLE assignments (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255),
  description TEXT,
  teacher_id BIGINT,
  due_date DATE,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);

-- Submissions
CREATE TABLE submissions (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  assignment_id BIGINT,
  student_id BIGINT,
  file_path VARCHAR(255),
  grade VARCHAR(10),
  submitted_at TIMESTAMP
);

```



Step-by-Step Guide

Step 1: Setup

- Install Laravel (composer create-project laravel/laravel cms-backend).
- Install React (npx create-react-app cms-frontend).
- Setup MySQL database and configure .env.

Step 2: Backend (Laravel)

- Create models: User, Content, Assignment, Submission.
- Setup migrations for schema above.

- Implement authentication (Laravel Sanctum).
- Build REST APIs:
 - /api/login, /api/register
 - /api/contents (CRUD)
 - /api/assignments (CRUD)
 - /api/submissions (CRUD)

Step 3: Frontend (React)

- Setup React Router (routes for Admin, Teacher, Student dashboards).
- Create components:
 - Login/Register
 - Content List & Editor
 - Assignment List & Submission Form
 - Notifications
- Use React Query/Axios for API calls.
- Role-based rendering (hide/show features).

Step 4: Integration

- Connect React frontend → Laravel API → MySQL DB.
- Test CRUD operations.
- Ensure role-based access works.

Step 5: Deployment (Optional Advanced)

- Dockerize services (React, Laravel, MySQL).
- Use Nginx reverse proxy.
- Deploy on VPS or cloud (AWS, DigitalOcean).



Suggested Project Timeline (6 Weeks)

Week	Focus	Key Tasks
Week 1	Project Setup	Install Laravel & React; Setup MySQL database; Create GitHub repo
Week 2	Backend Foundations	Create models & migrations; Implement authentication; Build user APIs
Week 3	Frontend Foundations	Setup React Router; Create login/register pages; Build dashboards
Week 4	Content Management	Implement CRUD for content; Add categories & status; Build React components
Week 5	Assignments & Submissions	Backend APIs for assignments/submissions; Teacher creates assignments, students submit; Grading & feedback system
Week 6	Finalization & Deployment	Polish UI/UX; Write documentation; Optional: Dockerize & deploy; Submit demo video/screenshots + GitHub repo



Deliverables

- Source code (GitHub repo).
- Database migrations & seeders.
- Documentation (README with setup instructions).
- Demo video/screenshots of working app.



Grading Criteria

Criteria	Weight	Details
Functionality	40%	Authentication, CRUD, role-based access
Code Quality	20%	Clean, modular, documented
UI/UX	20%	Clear dashboards, responsive design
Deployment/Extras	20%	Docker, CI/CD, advanced features